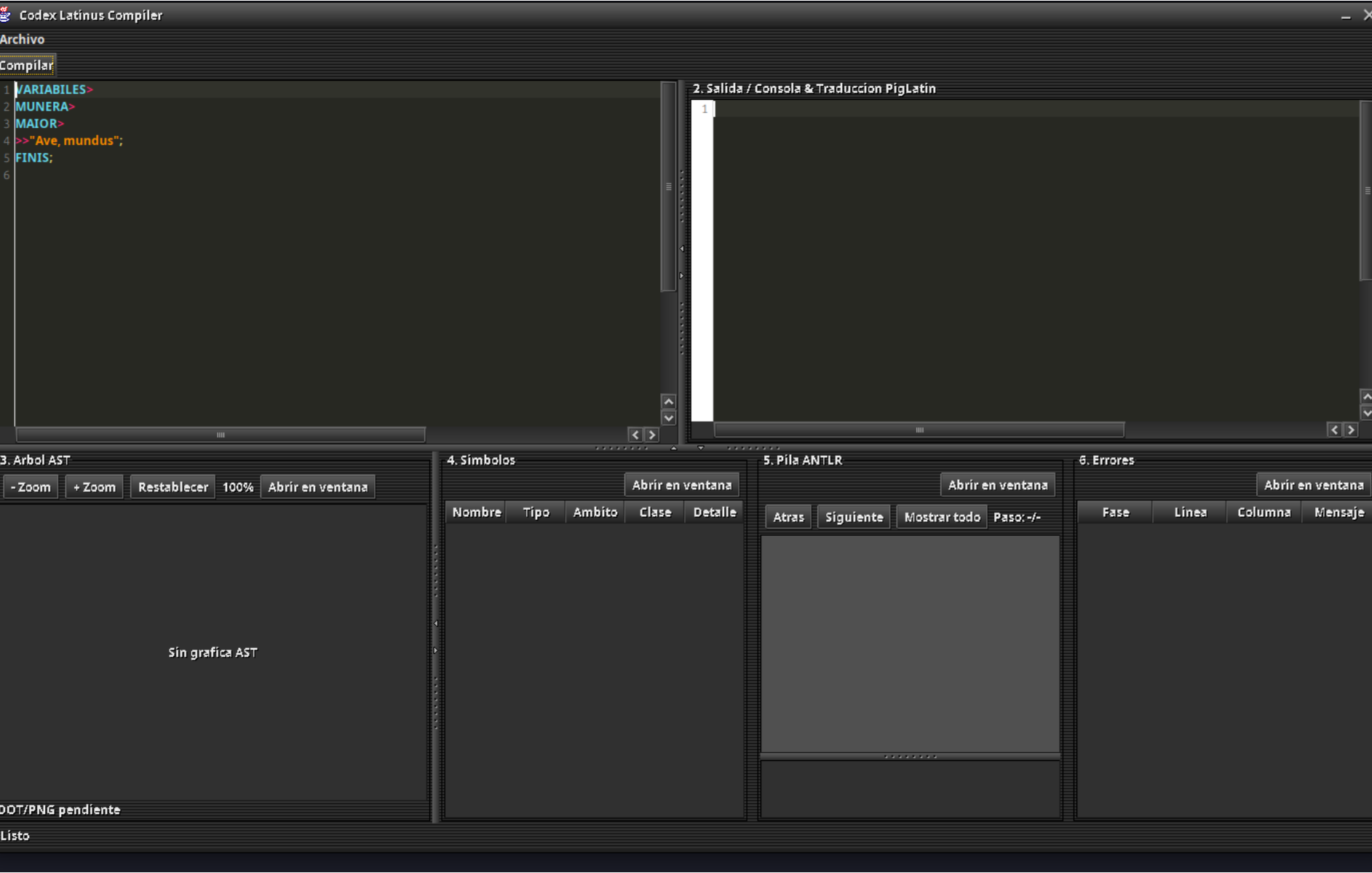


Manual de Usuario — IDE Codex Latinus & Traductor Pig Latin

1. Introducción

Bienvenido al **IDE Codex Latinus**, un entorno de desarrollo interactivo y compilador diseñado para la resistencia humana frente al Alto Mando Porcino. Esta herramienta permite redactar programas en el lenguaje clásico **Codex Latinus**, analizarlos en tiempo real para verificar su corrección léxica, sintáctica y semántica, inspeccionar el Árbol de Sintaxis Abstracta (AST), consultar la Tabla de Símbolos, monitorear la Pila del Parser y finalmente traducir el código al dialecto **Pig Latin** con exportación de archivos `.pig`.



2. Requisitos del Sistema e Instalación

2.1 Requisitos Mínimos

- **Sistema Operativo:** Linux (Ubuntu 20.04+, Debian, Fedora, Arch), Windows 10/11 o macOS 12+.
- **Java:** Java Development Kit (JDK) versión 21 o superior instalado y configurado en las variables de entorno (`JAVA_HOME`).
- **Graphviz (Recomendado):** Herramienta de renderizado de grafos (`dot`) instalada en el sistema para visualizar gráficamente el Árbol AST en formato PNG.
 - *Instalación en Linux (Debian/Ubuntu):* `sudo apt install graphviz`
 - *Instalación en Windows:* `winget install Graphviz.Graphviz` o descargar desde graphviz.org
 - *Instalación en macOS:* `brew install graphviz`
- **Resolución de Pantalla:** Mínimo 1366 × 768 píxeles (Recomendado 1920 × 1080).

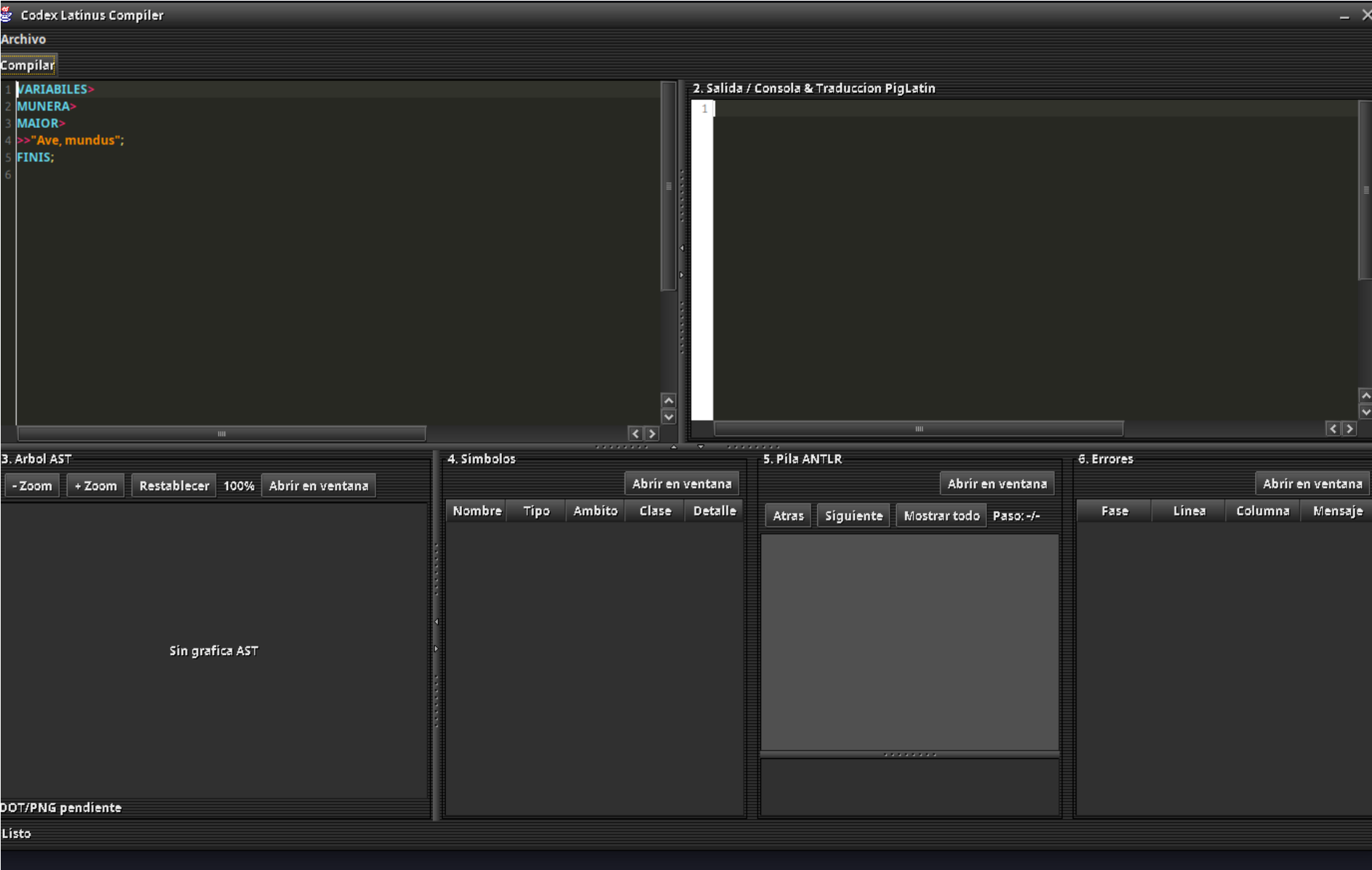
2.2 Ejecución del Entorno

Abra una terminal en la raíz del proyecto y ejecute:

```
./mvnw exec:java
```

3. Estructura de la Interfaz Gráfica

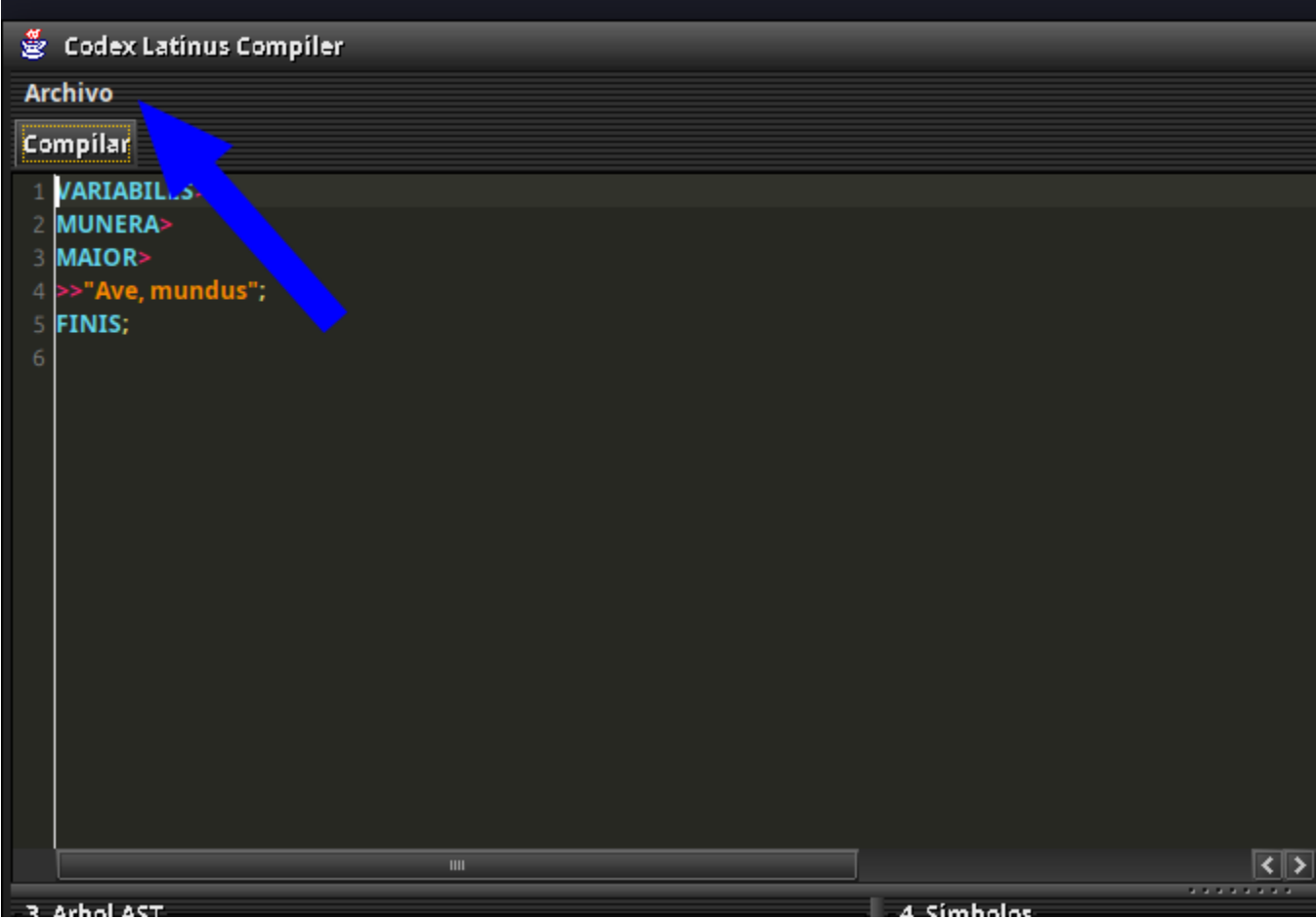
La interfaz gráfica del IDE está diseñada en un esquema moderno y modular organizado en **4 cuadrantes interactivos** con paneles divisores ajustables (*Split Panes*):



3.1 Barra de Menús y Barra de Herramientas

Ubicada en la parte superior de la ventana:

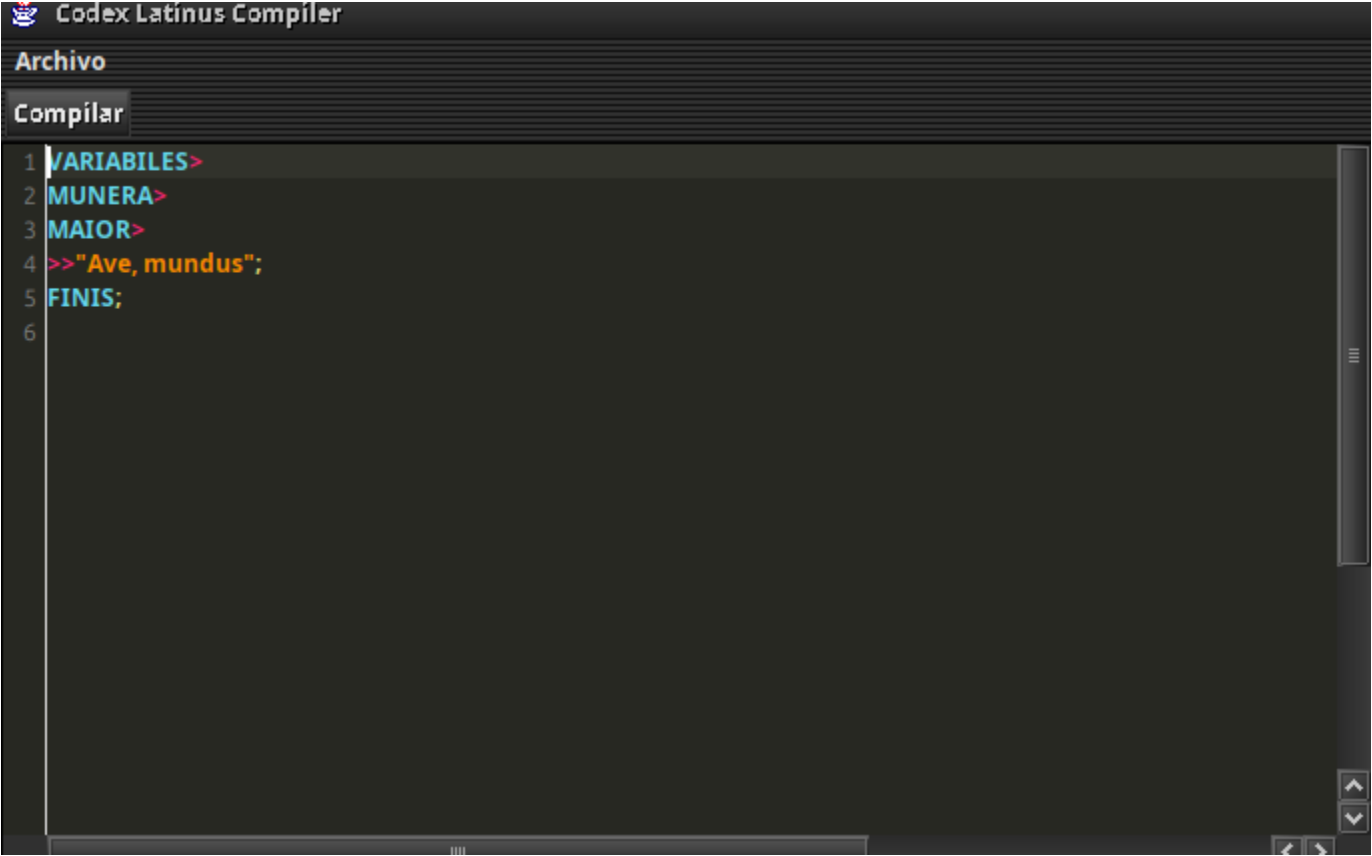
- **Menú Archivo:**
 - **Nuevo:** Limpia el editor y crea un nuevo espacio de trabajo.
 - **Abrir:** Abre un diálogo para cargar archivos fuente con extensión `.lat`.
 - **Guardar:** Guarda los cambios realizados en el archivo actual abierto.
 - **Exportar PigLatin:** Guarda la traducción generada en un archivo con extensión `.pig`.
-



3.2 Cuadrante 1: Editor de Código Fuente (.lat)

Ubicado en la parte superior izquierda. Proporciona:

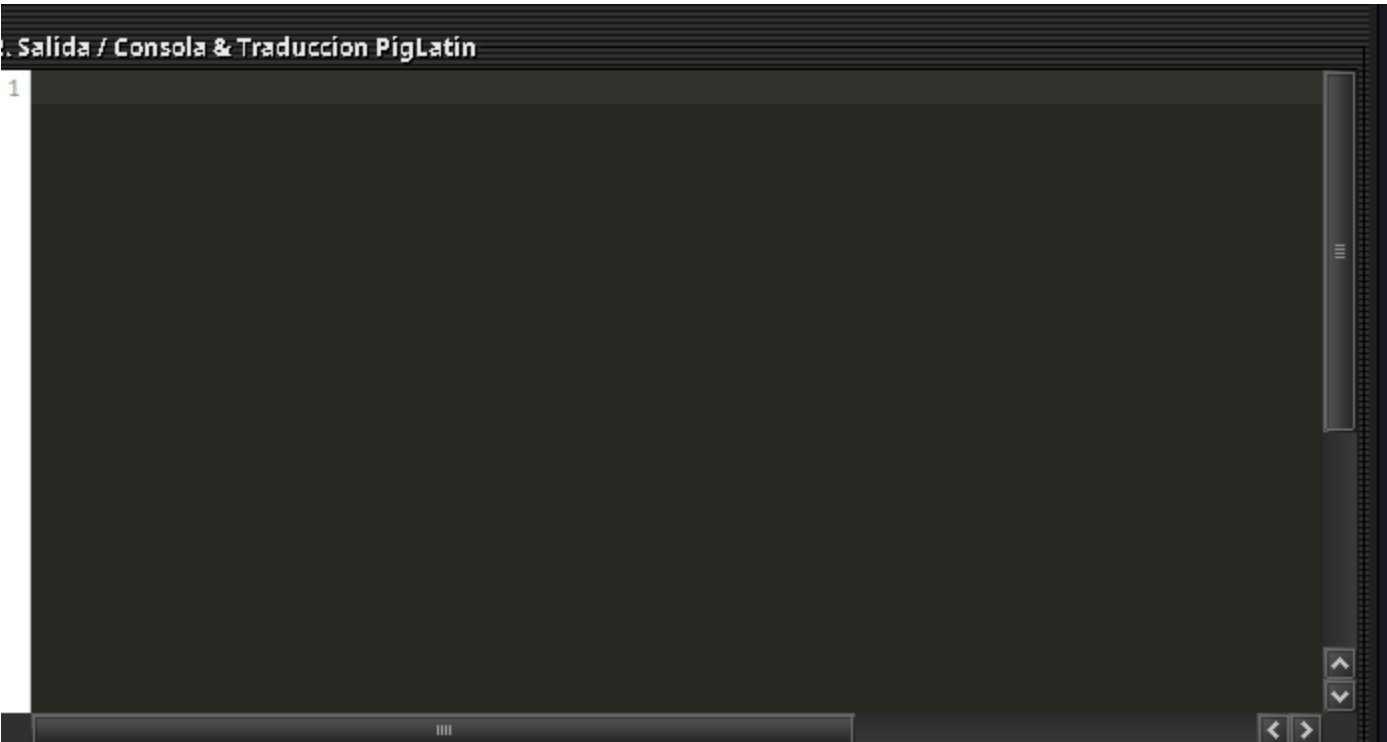
- **Coloreado de Sintaxis en Tiempo Real:** Reconocimiento de palabras reservadas (esto , ratio , actio , si , dum , per), tipos (numerus , textum , decimalis , etc.), cadenas, números y comentarios.
- **Numeración de Líneas:** Facilita la localización rápida de errores reportados por el compilador.
- **Tema Oscuro Monokai:** Diseñado para evitar fatiga visual durante sesiones prolongadas.



3.3 Cuadrante 2: Panel de Salida y Traducción Pig Latin

Ubicado en la parte superior derecha:

- Muestra el código traducido a **Pig Latin** automáticamente cuando el programa en latín es 100% válido y libre de errores.
- Resalta la sintaxis traducida (palabras transformadas como estoway , uerzafay , sentencias %OINK y %OINK_OINK).
- El contenido es de solo lectura para garantizar la integridad de la traducción generada por el AST.



3.4 Cuadrante 3: Visualizador del Árbol AST

Ubicado en la parte inferior izquierda:

- **Renderizado Gráfico PNG:** Muestra el diagrama visual del Árbol de Sintaxis Abstracta generado con Graphviz.
- **Controles de Zoom:**
 - **Botón** `+ Zoom` : Aumenta el tamaño del gráfico para ver detalles finos.
 - **Botón** `- Zoom` : Reduce el tamaño del gráfico para una vista panorámica.
 - **Botón** `Restablecer` : Devuelve el nivel de zoom al 100%.
 - **Indicador de Porcentaje:** Muestra la escala actual (ej. `120%`).
- **Botón** `Abrir en ventana` : Abre el árbol AST en una ventana maximizada independiente con barras de desplazamiento.



3.5 Cuadrante 4: Paneles de Diagnóstico y Monitoreo

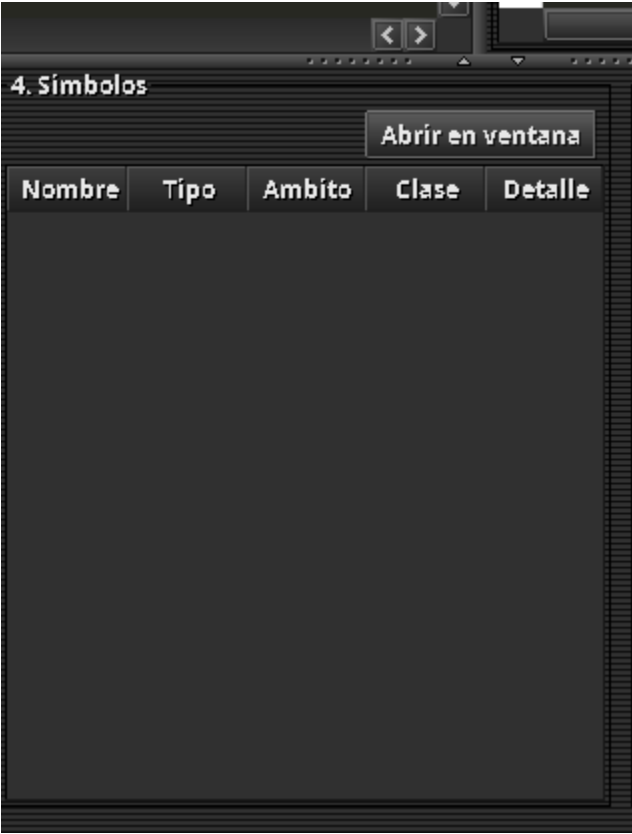
Ubicado en la parte inferior derecha, subdividido en 3 módulos interactivos:

A. Panel de Símbolos (Sub-panel 4)

Presenta una tabla estructurada con todos los identificadores reconocidos:

- **Nombre:** Nombre de la variable, función, estructura o arreglo.
- **Tipo:** Tipo de dato primitivo o nombre de la estructura.
- **Ámbito (Scope):** Ámbito donde fue declarado (`global` , nombre de función o bloque `for`).
- **Clase (Kind):** Clasificación (`VARIABLE` , `ARRAY` , `STRUCTURE` , `STRUCT_VARIABLE` , `FUNCTION`).
- **Detalle:** Información complementaria (dimensiones de arreglos, campos de estructuras, parámetros de funciones).

- **Botón** `Abrir en ventana` : Permite visualizar la tabla completa en una ventana emergente redimensionable.



B. Panel de Pila ANTLR (Sub-panel 5)

Permite observar paso a paso la simulación de la pila de reconocimiento sintáctico:

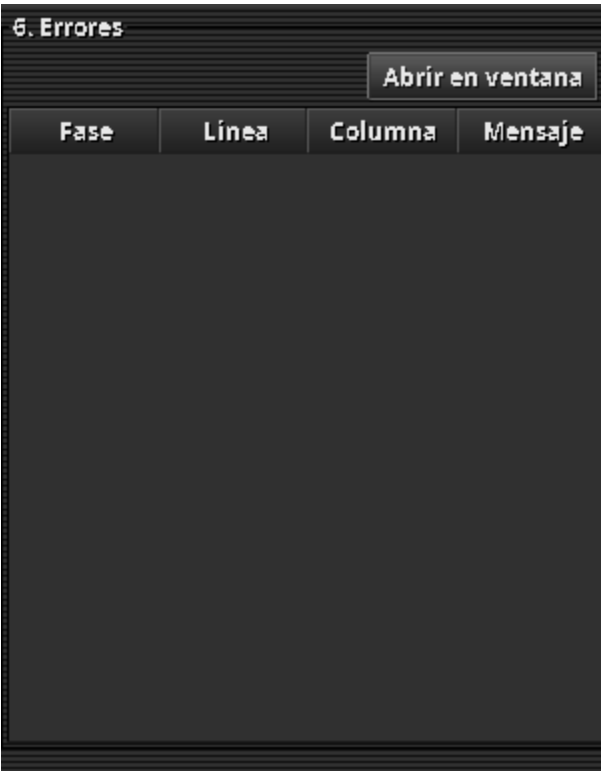
- **Controles de Navegación:**
 - **Botón** `Atrás ( < )`: Retrocede al snapshot de ejecución anterior.
 - **Botón** `Siguiente ( > )`: Avanza al siguiente snapshot de la pila.
 - **Botón** `Mostrar todo` : Alterna entre la vista paso a paso y la lista completa cronológica de operaciones `SHIFT` y `REDUCE` .
- **Área de Pila:** Muestra las reglas gramaticales apiladas actualmente (tope arriba).
- **Área de Registro (Log):** Detalla la operación realizada (`SHIFT` , `REDUCE` , `ACCEPT`), la regla involucrada y la profundidad de la pila.
- **Botón** `Abrir en ventana` : Abre el visor de pila en una ventana independiente.



C. Panel de Errores (Sub-panel 6)

Muestra el diagnóstico detallado de anomalías encontradas:

- **Columnas:** `Fase` (`LEXICA` , `SINTACTICA` , `SEMANTICA`), `Línea` , `Columna` y `Mensaje` .
- **Navegación al Código con Clic:** Al seleccionar o hacer doble clic en una fila de error, el cursor del editor salta automáticamente a la línea y columna exactas donde se originó el problema.
- **Botón** `Abrir en ventana` : Abre la lista de errores en una ventana flotante para análisis cómodo.



3.6 Barra de Estado Inferior

Indica en todo momento el estado actual del compilador: `Listo` , `Compilando...` , `Compilación exitosa` , `Compilación con X error(es)` o el nombre del archivo guardado/exportado.

4. Guía de Uso Paso a Paso

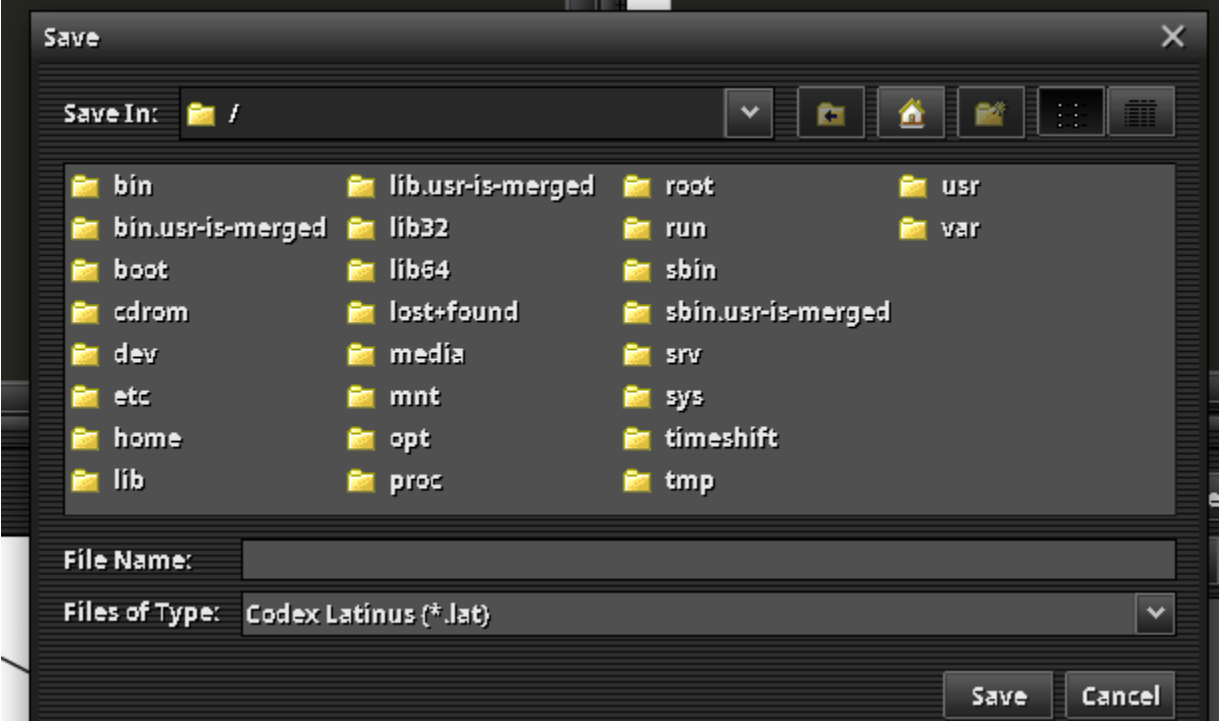
4.1 Caso de Uso 1: Crear y Editar un Archivo Fuente

- Al abrir la aplicación, se carga una plantilla inicial básica en el editor.
- Si desea empezar un código desde cero, haga clic en el menú `Archivo -> Nuevo` .
- Escriba su código en latín respetando la estructura obligatoria:

```
VARIABLES>
esto municiones : numerus 50;
esto listo : verum;

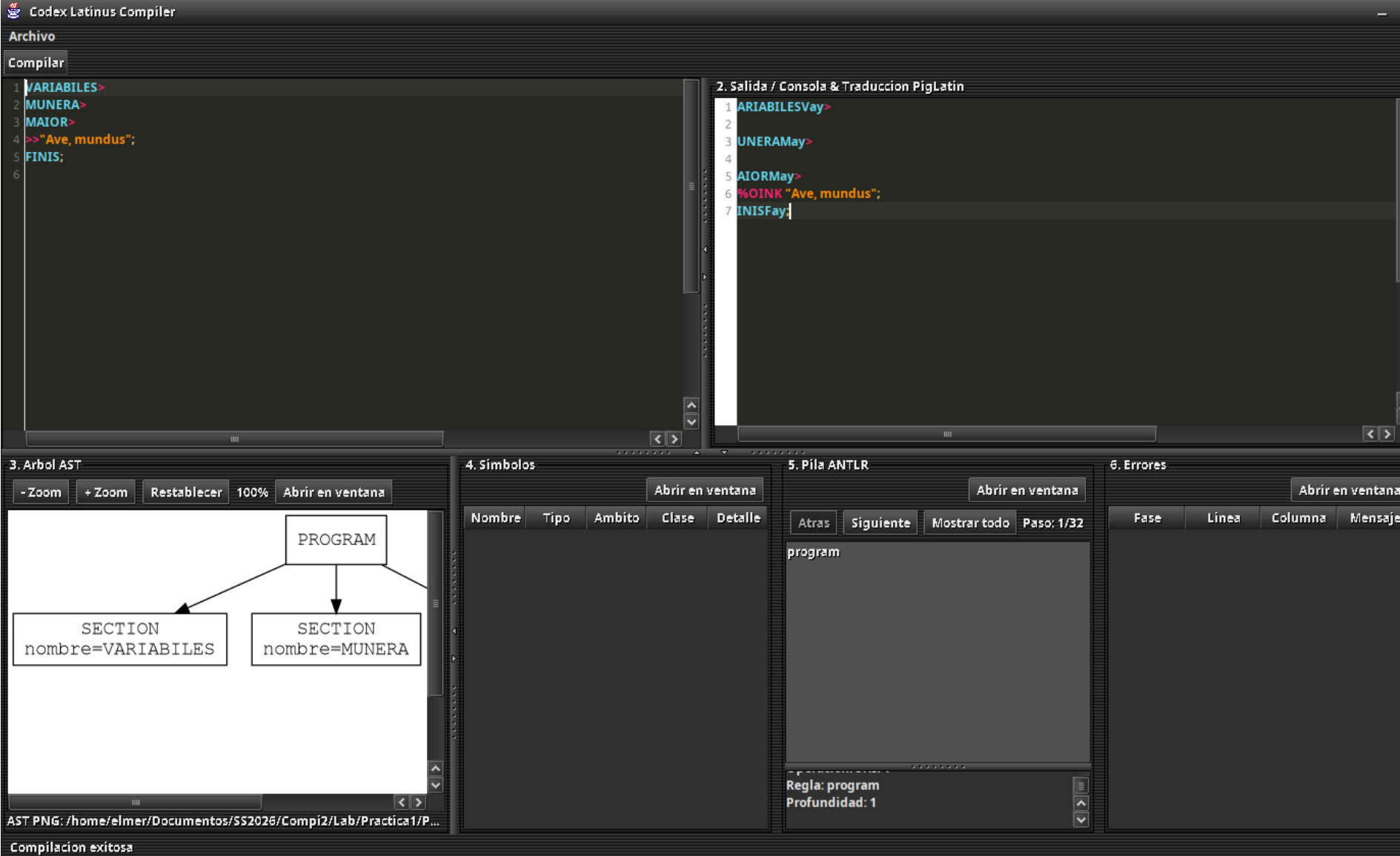
MAIOR>
>> "Preparando ataque porcino...";
>> municiones;
FINIS;
```

- Guarde su archivo seleccionando `Archivo -> Guardar` (se guardará con extensión `.lat`).



4.2 Caso de Uso 2: Compilar y Traducir a Pig Latin

1. Una vez ingresado el código en el editor, presione el botón `Compilar` en la barra superior.
2. Si el código no contiene errores:
 - La barra de estado mostrará: `Compilación exitosa`.
 - En el panel superior derecho aparecerá inmediatamente la traducción a **Pig Latin**.
 - El panel inferior izquierdo dibujará el gráfico del **Árbol AST**.
 - La **Tabla de Símbolos** se poblará con las variables y funciones.
 - La **Pila ANTLR** contendrá la traza completa de derivación sintáctica.



4.3 Caso de Uso 3: Exportar el Código Traducido (.pig)

1. Tras una compilación exitosa, diríjase al menú `Archivo -> Exportar PigLatin`.
2. Seleccione la carpeta de destino y el nombre del archivo.
3. El sistema creará el archivo `.pig` con el código listo para infiltrar en los centros de datos porcinos.
4. La barra de estado confirmará: `PigLatin exportado: <archivo>.pig`.

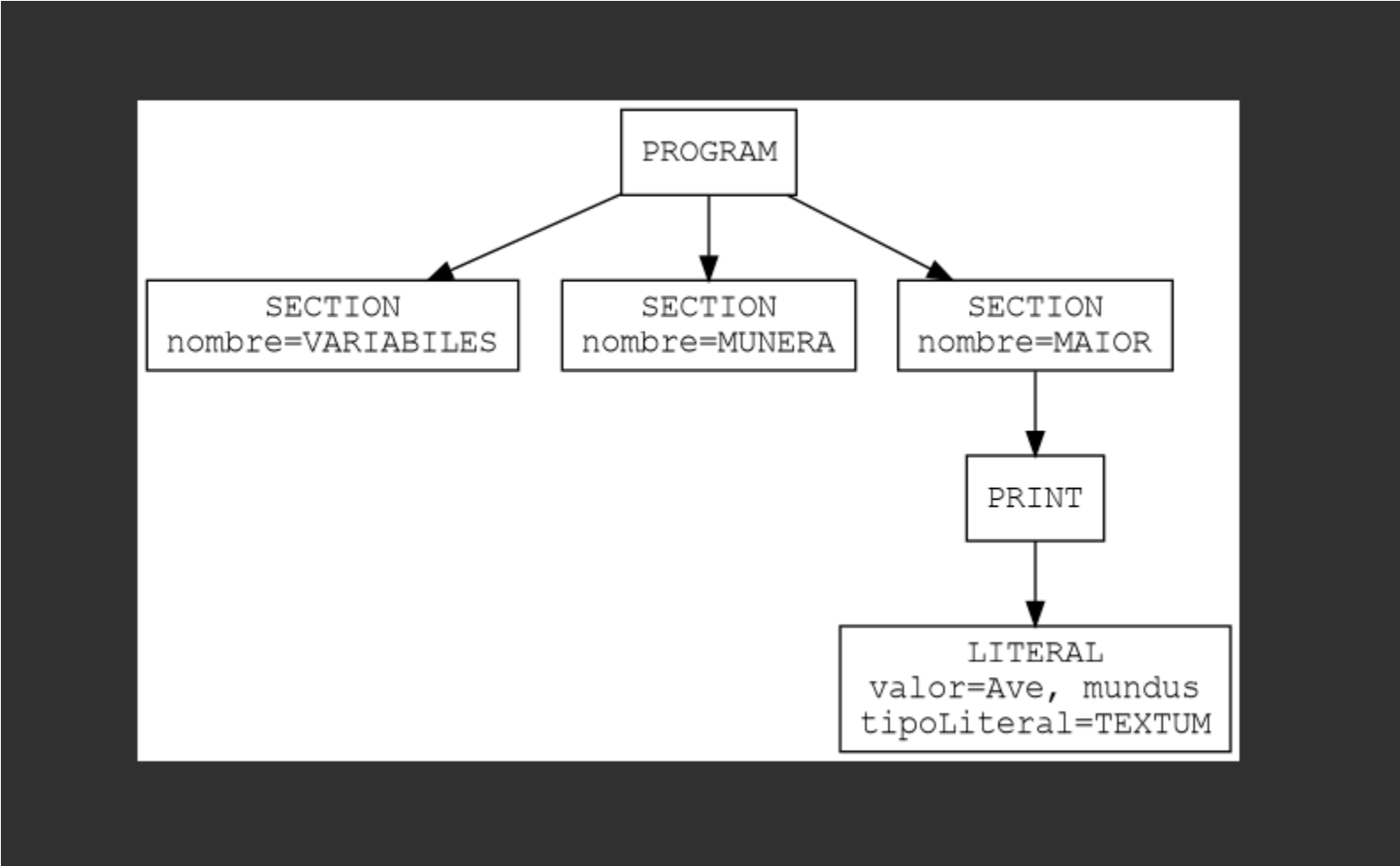


4.4 Caso de Uso 4: Navegar e Inspeccionar Errores

1. Si introduce un error intencional (por ejemplo, declarar `esto x : numerus "hola";` o una condición no booleana `si (10) { ... }`):
2. Presione el botón `Compilar` .
3. La barra de estado indicará: `Compilación con X error(es)` .
4. El panel de salida PigLatin permanecerá limpio para evitar la emisión de código corrupto.
5. En el panel **6. Errores**, haga clic sobre el error listado.
6. El editor colocará instantáneamente el cursor en la línea y columna del fallo para su rápida corrección.

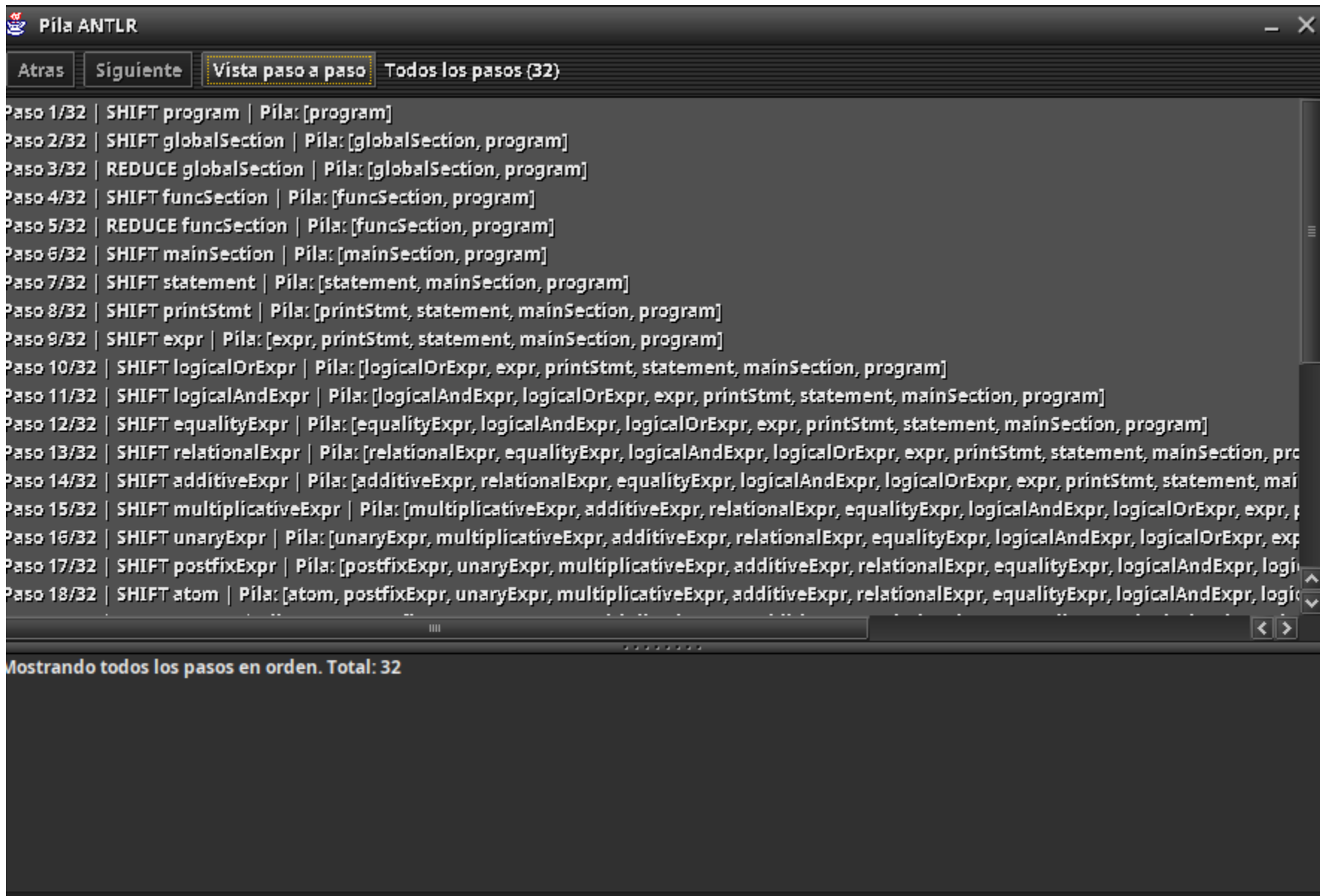
4.5 Caso de Uso 5: Explorar el Árbol AST con Zoom

1. Con un programa válido compilado, observe el panel **3. Árbol AST**.
2. Utilice el botón `+ Zoom` para acercar los nodos y leer claramente los atributos (`operador` , `tipoDeclarado` , `valor`).
3. Utilice el botón `- Zoom` para alejar y tener una perspectiva global de la estructura del árbol.
4. Haga clic en `Abrir en ventana` para abrir una vista en pantalla completa del AST.



4.6 Caso de Uso 6: Inspección de Pila ANTLR Paso a Paso

1. En el panel **5. Pila ANTLR**, observe el contador de pasos (ej. `Paso: 1/48`).
2. Presione el botón `Siguiente` sucesivamente para ver cómo el parser ingresa a las reglas (`SHIFT`) y cómo reduce las expresiones (`REDUCE`).
3. Si desea ver el historial completo de una sola vez, presione el botón `Mostrar todo` .



5. Ejemplos de Programas en Codex Latinus

Ejemplo 1: Operaciones Aritméticas y Condiciones

```
VARIABLES>
esto a : numerus 15;
esto b : numerus 4;
esto resultado : decimalis 0.0;

MAIOR>
resultado = a / b;
>> "El resultado decimal es:";
>> resultado;

si (resultado > 3.0) {
    >> "Operacion satisfactoria";
} aliter {
    >> "Operacion bajo el umbral";
} finis;
FINIS;
```

Ejemplo 2: Estructuras y Arreglos

```
VARIABLES>
structura Soldado {
    esto nombre : textum,
    esto rango : numerus
} finis;

esto capitán : Soldado {
    nombre: "Marcus",
    rango: 5
};

series escuadrón[3] : numerus {101, 102, 103};

MAIOR>
>> capitán.nombre;
>> escuadrón[0];
FINIS;
```

Ejemplo 3: Funciones `ratio` (con retorno) y `actio` (void)

```
VARIABLES>
esto base : numerus 8;
esto altura : numerus 5;

MUNERA>
ratio numerus calcularArea(esto b : numerus, esto h : numerus) {
    VARIABLES[
        esto area : numerus 0;
    ]
    area = (b * h) / 2;
    reddere area;
} finis;

actio alertarResistencia() {
    >> "¡Alerta! Patrulla de cerdos detectada.";
} finis;

MAIOR>
alertarResistencia();
>> "Area calculada:";
>> calcularArea(base, altura);
FINIS;
```

6. Preguntas Frecuentes y Solución de Problemas

Pregunta / Problema	Causa Probable	Solución
El panel AST dice "Graphviz (dot) no disponible".	El binario <code>dot</code> de Graphviz no está instalado o no se encuentra en la variable de entorno <code>PATH</code> .	Instale Graphviz en su sistema operativo (<code>sudo apt install graphviz</code> en Linux o instalador oficial en Windows) y reinicie el IDE.
No se genera el código en Pig Latin tras compilar.	El programa contiene errores léxicos, sintácticos o semánticos.	Revise el panel 6. Errores , haga clic en las filas de error para corregir el código en el editor y vuelva a presionar Compilar .
¿Por qué las lecturas con <code><<</code> no llevan punto y coma?	De acuerdo a la especificación oficial de Codex Latinus, las sentencias de lectura <code><<</code> no requieren <code>;</code> .	Escriba <code>nombre <<</code> o <code><<</code> directamente sin agregar <code>;</code> al final.
Error: "Corrupción de Flujo: la condición debe ser BOOL".	Se colocó una expresión numérica o de texto dentro de un <code>si</code> , <code>dum</code> o <code>per</code> .	Asegúrese de utilizar operadores relacionales o lógicos (<code>==</code> , <code>!=</code> , <code><</code> , <code>></code> , <code>&&</code> , <code>`</code>
Error: "La función ratio no retorna en todos los caminos".	Una función declarada con <code>ratio</code> tiene ramas de control donde no se ejecuta la sentencia <code>reddere</code> .	Verifique que todas las bifurcaciones <code>si / aliter</code> y el final del cuerpo de la función posean un retorno válido.